

Processi e Thread

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

PROCESSI

Cap. 1: par. 1.5.1, 1.5.2, 1.6.1

Cap. 2: par. 2.1

Definizioni preliminari

Programma

```
void X(int b){
    if(b == 1){
        ...
        ...
    }

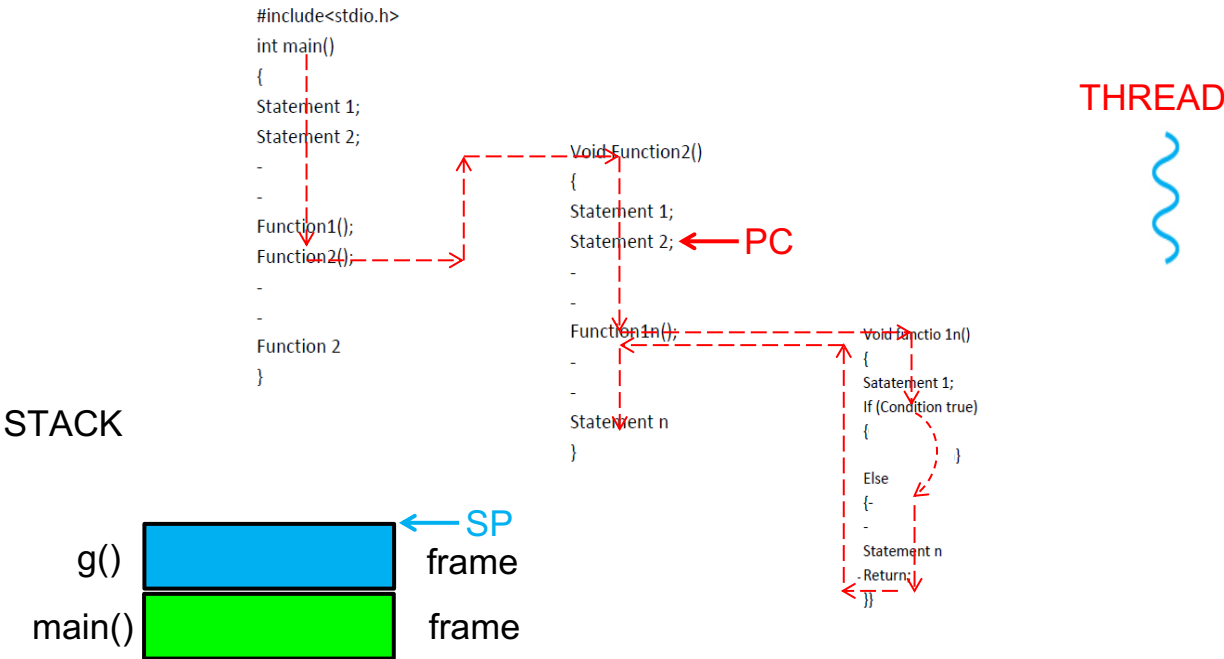
    int main(){
        int a = 2;
        X(a);
    }
}
```

Programma
(eseguibile)

```
Contenuto della sezione .text:
1060 f30f1efa 31ed4989 d15e4889 e24883e4 ....1.I..^H..H..
1070 f050544c 8d056601 0000488d 0def0000 .PTL..f...H....
1080 00488d3d c1000000 ff15522f 0000f490 .H.=.....R/....
1090 488d3d79 2f000048 8d05722f 00004839 H.=y/..H..r/..H9
10a0 f8741548 8b052e2f 00004885 c07409ff .t.H.../..H..t..
10b0 e00f1f80 00000000 c30f1f80 00000000 .....
10c0 488d3d49 2f000048 8d35422f 00004829 H.=I/..H.5B/..H)
10d0 fe4889f0 48c1ee3f 48c1f803 4801c648 .H..H..?H...H..H
10e0 d1fe7414 488b0505 2f000048 85c07408 ..t.H.../..H..t.
10f0 ffe0660f 1f440000 c30f1f80 00000000 ..f..D.....
1100 f30f1efa 803d052f 00000075 2b554883 .....=./...u+UH.
1110 3de22e00 00004889 e5740c48 8b3de62e =.....H..t.H.=..
1120 0000e819 ffffffff 64ffffff c605dd2e .....d.....
1130 0000015d c30f1f00 c30f1f80 00000000 ...].....
1140 f30f1efa e977ffff fff30f1e fa554889 .....w.....UH.
1150 e5488d3d ac0e0000 e8f3feff ffb80000 .H.=.....
```

objdump -s hello

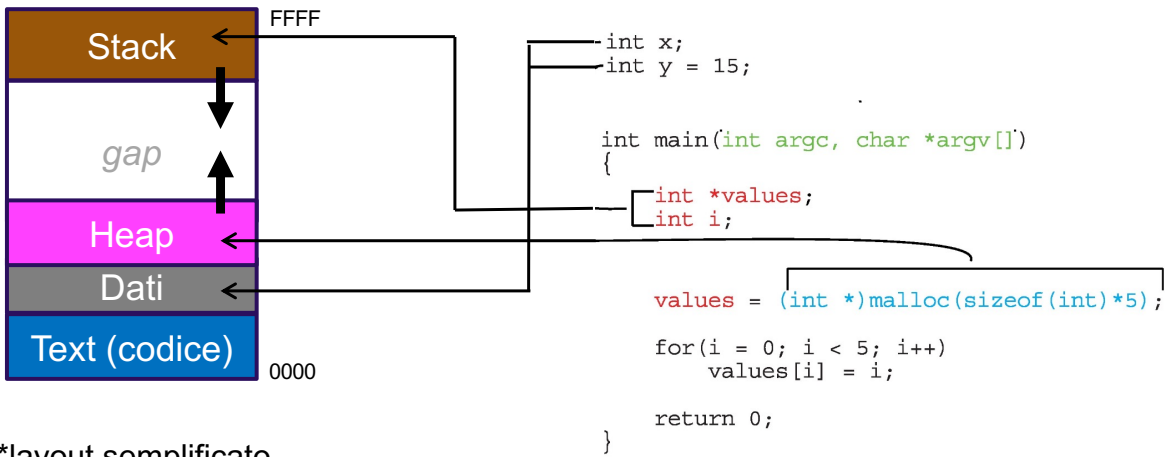
Programma objdump (mostra informazioni dai file oggetto):
<https://linux.die.net/man/1/objdump>



Processo: programma in esecuzione

spazio degli indirizzi*

locazioni di memoria da 0
a un massimo



*layout semplificato

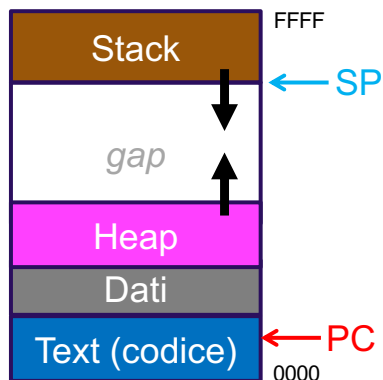
Prospetto riepilogativo

Classe di memorizzazione variabili	Definizione	Visibilità	Allocazione
Automatiche	Definite in un blocco	Blocco	Area <i>stack</i>
Esterne	Definite all'esterno di ogni blocco	Tutto il programma	Area dati statici
Automatiche statiche	Definite in un blocco + <i>static</i>	Blocco	Area dati statici
Register	Definite in un blocco + <i>register</i>	Blocco	Se possibile, in un registro del processore altrimenti in area <i>stack</i>
Dinamiche	Tramite puntatori e operatori <i>new/delete</i>	Finché esiste un riferimento valido	Area <i>heap</i>

Processo: programma in esecuzione

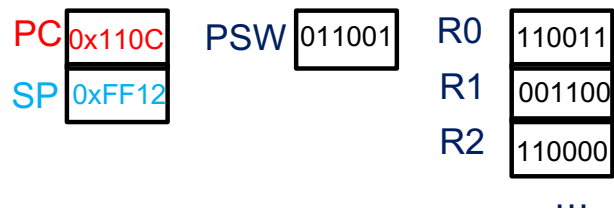
spazio degli indirizzi*

locazioni di memoria da 0
a un massimo



*layout semplificato

Registri



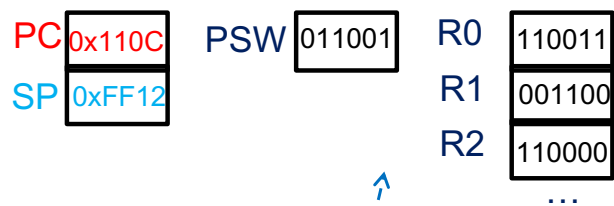
Alcune informazioni che kernel mantiene in memoria per ciascun processo:

- identificativo;
- CPU context (PC, SP, PSW, registri, ...);
- file aperti;
- stato;
- scheduling;
- ...

Processo: programma in esecuzione

- salvataggio in memoria nello spazio* dedicato al processo P_i

Registri (i veri registri CPU)

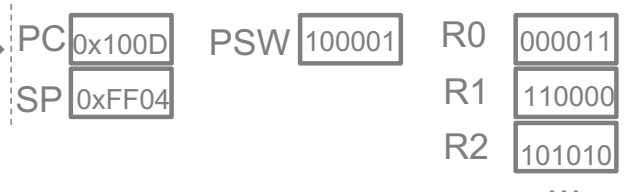


parte del PCB di P_i



non sono registri CPU

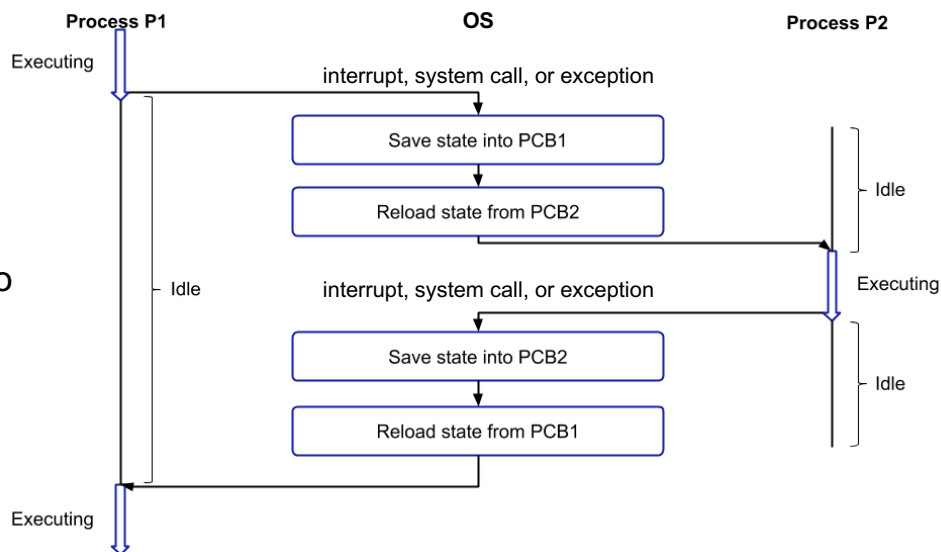
- caricamento dallo spazio* in memoria dedicato a P_j



* **Process Control Block** (PCB) o descrittore del processo; l'insieme dei PCB forma la **tabella dei processi**.

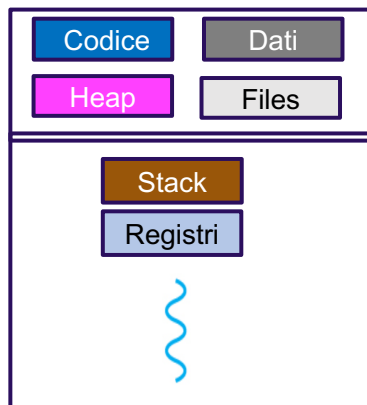
Context switch

In generale, ogni volta che il SO acquisisce il controllo della CPU, esso ha la possibilità di eseguire un **context switch**.



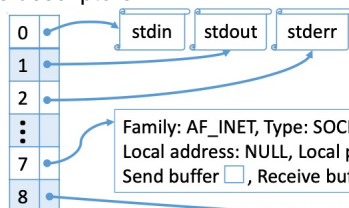
Processo

- Processo: astrazione di **programma in esecuzione**:
 - spazio degli indirizzi e thread di controllo;
 - risorse (per es., i **file**);
 - strutture dati usate dal kernel per rappresentarlo (**PCB**).



*by default every process starts with three "files":

file descriptors



Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment info Pointer to data segment info Pointer to stack segment info	Root directory Working directory File descriptors User ID Group ID

Process Control Block (PCB)

*UNIX: everything is a **file** (sockets, devices, hard drives, printers, pipes, ...).

**In figura, 'registri' non sono i registri fisici della CPU.

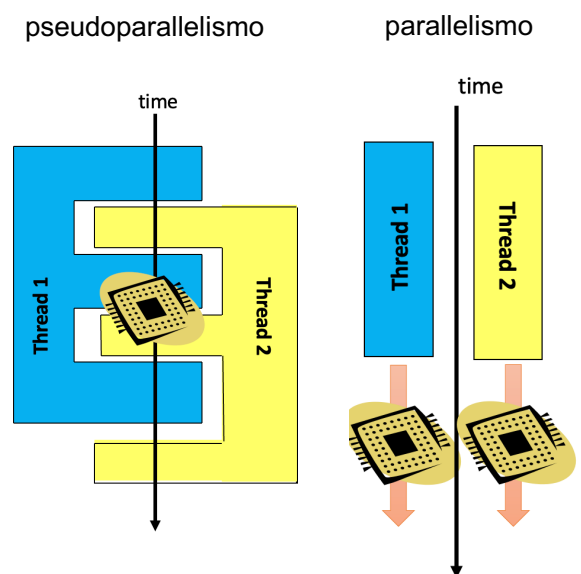
Process Control Block (PCB)

- Un PCB per processo:
 - tutta l'informazione riguardante il processo (a parte i contenuti dello spazio degli indirizzi);
 - l'insieme dei PCB è detto **tabella dei processi**.

Process management	Memory management	File management
Registers	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Program status word	Pointer to stack segment info	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

Implicazioni pratiche*

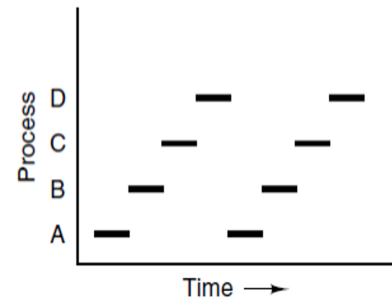
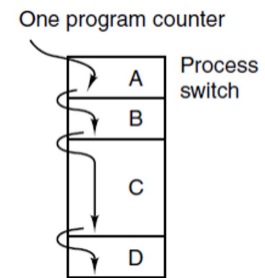
- Meccanismo alla base della **multiprogrammazione**;
- Il PC (program counter) *fisico* è unico;
- La CPU esegue **1** processo alla volta e "passa" avanti/indietro tra i processi:
 - *illusione* di parallelismo, ossia **pseudo-parallelismo**, per i sistemi **monoprocessore**;
 - **vero e proprio** parallelismo nei sistemi **multiprocessore**.



*NB: assunzione di singola CPU; se esistono n CPU (core) **ciascuna** eseguirà 1 processo alla volta.

Implicazioni pratiche*

- Tutti i processi fanno qualche "progresso", ma solo uno è effettivamente attivo;
- Processi differenti possono eseguire più istanze dello stesso programma;
- I processi sono **indipendenti**:
 - per farli **interagire** con altri processi, bisogna ricorrere a **meccanismi espliciti**.
- *In generale*, il SO **non offre** garanzie temporali o di ordinamento.

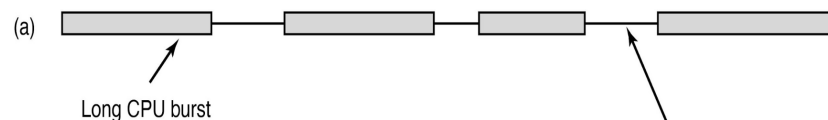


*NB: assunzione di singola CPU; se esistono n CPU (core) **ciascuna** eseguirà 1 processo alla volta.

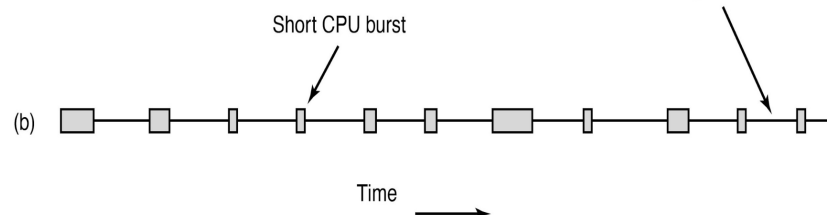
Nota prima di procedere*

- Tutti i processi alternano **burst** (fasi intense) di CPU e I/O

Processo
CPU bound

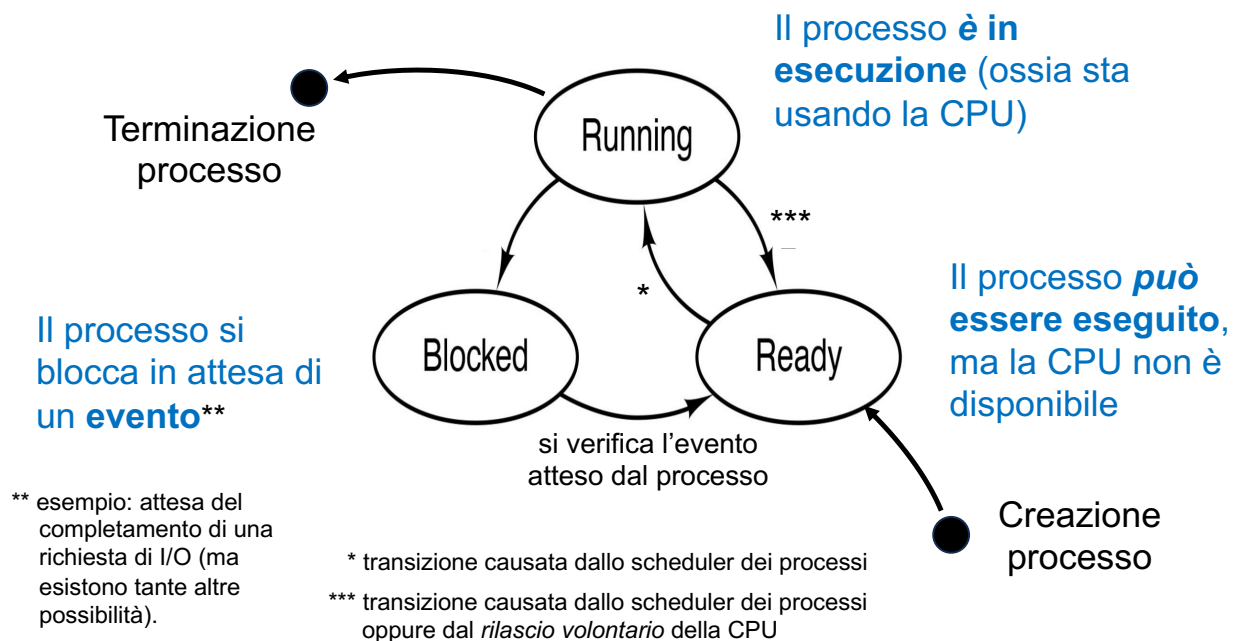


Processo
I/O bound



*Estratto dal par. 2.5.1 "Comportamento dei processi"

Stati di un processo



Creazione di un processo

- **"Quando"** sono creati i processi:
 - *durante* l'avvio del sistema operativo;
 - *dopo* l'avvio del SO, dai processi in esecuzione;
 - su richiesta/azione dell'utente del SO:
 - per es., doppio click o avvio di un job (sistemi batch).
- Un processo in esecuzione esegue una specifica **chiamata di sistema** per la *creazione* di un processo:
 - per es., in UNIX, tale chiamata è **fork**:
 - unico modo in cui un processo UNIX può essere creato, fatta eccezione per il processo **init** (o **systemd** Linux) creato, invece, all'avvio del SO.

Terminazione di un processo

- Terminazione di un processo:
 - uscita **normale** (il processo ha finito "naturalmente" il proprio lavoro);
 - chiamata di sistema **exit** in UNIX;
 - uscita su **errore**, tipicamente due casi:
 - per es., errore dovuto ad input mancante o errato;
 - errore causato dal processo (per es., div. per zero);
 - terminazione effettuata da un altro processo:
 - chiamata di sistema **kill** in UNIX;

Transizione Running→Ready

- Un processo transita da **running** a **ready** se:
 - rilascia **volontariamente** la CPU (operazione utile in caso di scheduling **non preemptive** o 'senza prelazione');
 - il processo in esecuzione invoca una specifica system call (per es. **sched_yield**) per rilasciare la CPU;
 - per esempio, scade il **quanto di tempo** ad esso assegnato (in caso di scheduling **preemptive**, ossia 'con prelazione'):
 - è necessario un **timer interrupt*** che consenta al SO e, quindi allo scheduler, di intervenire;
 - gli scheduler moderni sono preemptive.

* tipicamente, nel caso **non preemptive** durante un timer interrupt non sono prese decisioni di scheduling.

Esempi di transizioni di stato (semplificati)

SYSTEM CALL

- il processo P_Y è **RUNNING**;
- P_Y richiede una lettura dal disco;
- il SO prende il controllo della CPU;
- ...
- il SO decide di effettuare un **context switch**:
 - salva il contesto di P_Y in PCB_Y ;
 - imposta lo stato di P_Y a **BLOCKED**;
 - inserisce PCB_Y nel **blocked set**;
 - preleva PCB_x dal **ready set**;
 - imposta lo stato di P_x a **RUNNING**;
 - carica il contesto di P_x da PCB_x ;
- il processo P_x è ora **RUNNING**.

CLOCK INTERRUPT

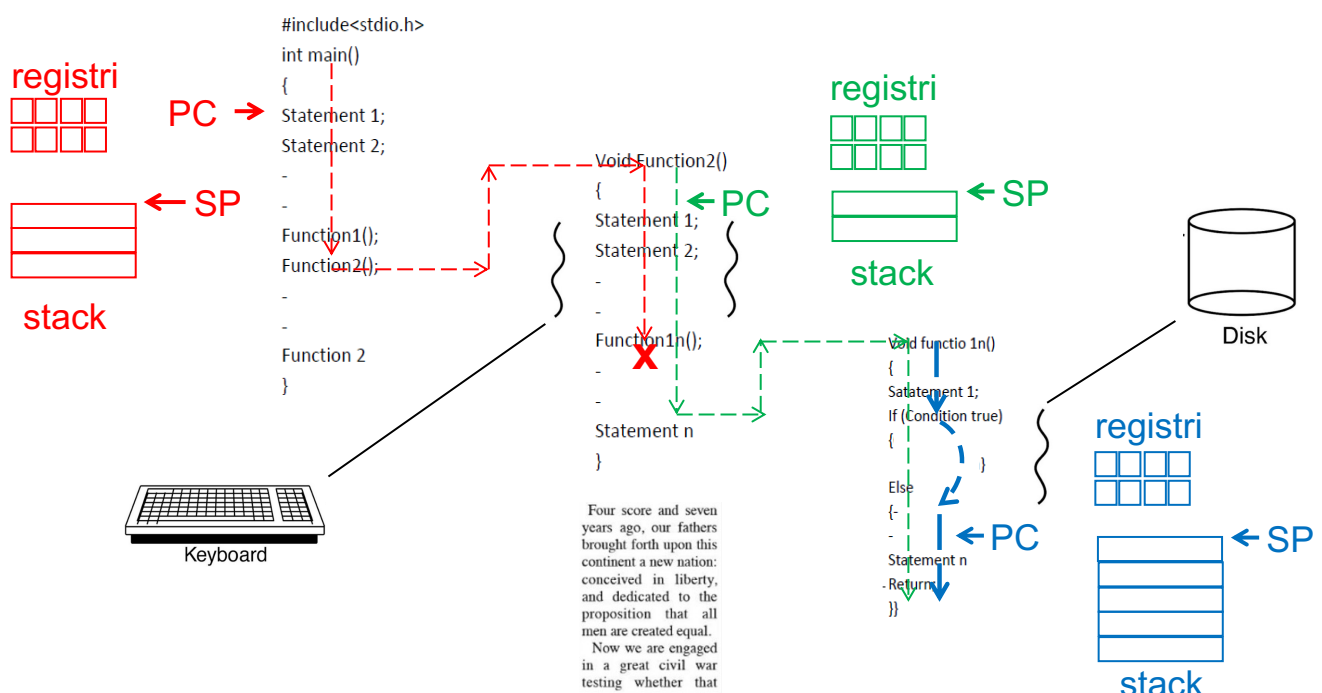
- il processo P_x è **RUNNING**;
- l'**HW** genera un clock interrupt;
- il SO prende il controllo della CPU;
- ...
- il SO decide di effettuare un **context switch** (se il *quanto* di P_x è scaduto):
 - salva il contesto di P_x in PCB_x ;
 - imposta lo stato di P_x a **READY**;
 - inserisce PCB_x nel **ready set**;
 - preleva PCB_Y dal **ready set**;
 - imposta lo stato di P_Y a **RUNNING**;
 - carica il contesto di P_Y da PCB_Y ;
- il processo P_Y è ora **RUNNING**.

Riferimenti ai comandi/segnali trattati

- **ps** (process status): snapshot dei processi correnti:
 - esempi di invocazione: **ps aux** (BSD-like), **ps -ef**, **ps -efL**;
 - **e**=tutti i processi; **f**=full format; **L**=mostra i thread;
 - <https://man7.org/linux/man-pages/man1/ps.1.html>
- **ps tree**: albero dei processi:
 - esempi di invocazione: **ps tree -p -s PID**;
 - **p**=mostra pid; **s**=mostra i genitori del processo specificato;
 - <https://man7.org/linux/man-pages/man1/ps tree.1.html>
- **kill PID** / **kill -9 PID**: terminazione processo;
- **Ctrl+Z**: sospensione processo / **Ctrl+C**: terminazione processo
- **fg**: mette un processo in *foreground*.

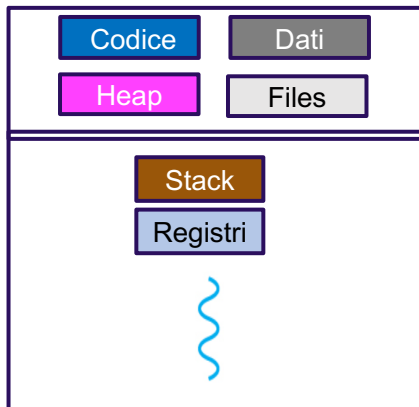
THREADS

Cap. 2: par. 2.2 (escluso 2.2.7)

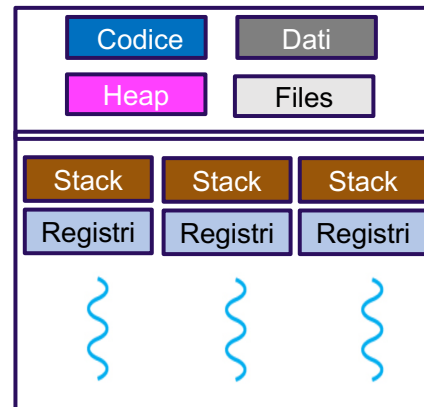


Aspetto chiave

- Separazione tra **esecuzione** (thread di controllo, o semplicemente thread) e **possesso di risorse**:
 - thread**: flusso di controllo sequenziale in un processo.



single threaded process

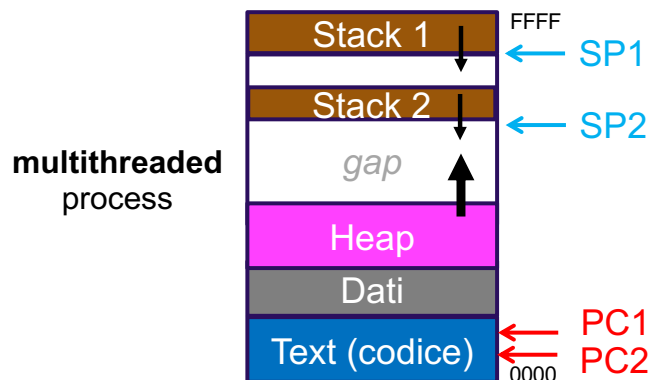
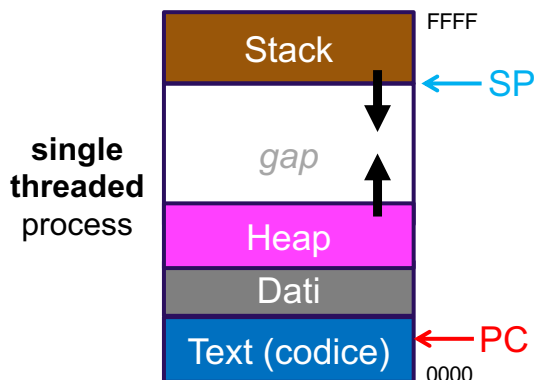


multithreaded process

Come appare ora lo spazio degli indirizzi

spazio degli indirizzi*

locazioni di memoria da 0
a un massimo



*layout semplificato

Processo multithreaded

- Un **unico** processo di elaborazione, ma in grado di eseguire **più thread** in maniera concorrente;
- Ciascun thread ha un proprio PC, stack, registri e **stato**.

elementi **condivisi**
tra tutti i thread del
processo

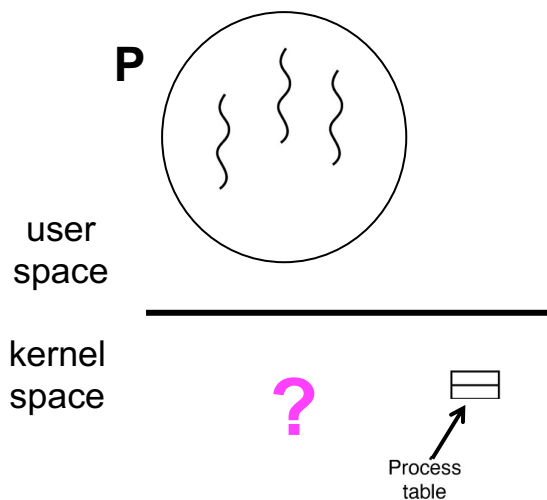
Per process items	Per thread items
Address space Global variables Open files Child processes Pending alarms Signals and signal handlers Accounting information	Program counter (PC) Registers Stack State*

elementi **privati**
dei thread

* gli **stati di un thread** sono gli stessi già descritti per i processi

Alcune considerazioni

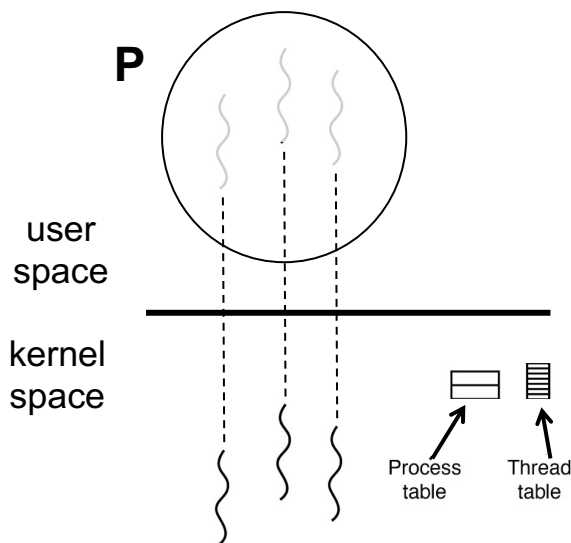
- I thread sono anche noti come *lightweight process*;
- La **creazione/terminazione** di un thread è più efficiente della creazione di un processo;
- Il **context switch** tra threads ha decisamente un minor overhead di quello tra processi;
- La **comunicazione** tra threads è più semplice di quelle tra processi poiché non coinvolge il kernel;
- Thread *diversi* di un processo **non sono** indipendenti quanto lo sono processi *diversi*:
 - non c'è **protezione** fra i thread;
 - paradigma di programmazione complesso.



Aspetti implementativi

- Il SO è necessariamente tenuto a "sapere" dell'esistenza dei thread del processo?
 - **kernel-level** thread vs **user-level** thread;
 - **notizia**: è possibile avere un processo multithreaded anche su SO che *non* supportano i thread.
- Altro aspetto: che succede dopo una **fork**?
 - si *clonano* tutti i thread o solo il chiamante?
- Ancora, gestione dei segnali:
 - segnalazione a tutti i thread o al singolo thread;
 - problema dei signal handlers (per thread o per processo?).

Kernel-level thread



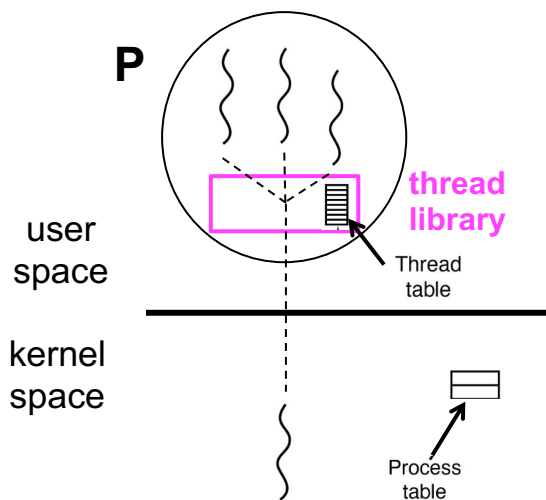
- Il kernel è a conoscenza dei thread del processo:
 - fornisce le system call necessarie per la gestione dei thread (per es., **clone**, in Linux);
- Il kernel mantiene, per i thread, un sottoinsieme delle informazioni tipicamente usate per i processi;
- I thread sono soggetti allo **scheduling** operato dal **SO**:
 - se un thread va in **blocked**, il kernel potrà schedare un altro thread.

Sidenote: implementazione thread Linux*

- In Linux, ogni processo è rappresentato dalla struttura **task_struct**, così come ogni thread:
 - un processo a singolo thread ha una **task_struct**; un processo multithread ha una **task_struct** per ogni thread;
 - un thread appare al SO come un "normale" processo, ma che condivide delle risorse con altri processi.
- System call **clone** di Linux* (introdotta nel 2000):
 - può creare un thread nel processo corrente o in uno nuovo;
 - consente di specificare quali risorse debbano essere condivise con il chiamante.

*Paragrafo 10.3.3 del libro di testo

User-level thread



- Il kernel non sa nulla dei thread del processo:
 - Il kernel "vede" il consueto processo *ordinario*, a singolo thread.
- I thread sono implementati tramite una **thread library**, che gestisce i thread.
- Le informazioni sui thread sono gestite dalla libreria.
- Lo **scheduling** è gestito dalla thread library, che decide quale thread mandare in esecuzione quando il processo diventa running.

User-level thread (vantaggi)

- Minore overhead per il context switch (che non richiede il passaggio in kernel mode);
- Scheduler dei thread indipendente da quello dei processi;
- Thread user-level possono essere implementati anche su SO che *non* supportano i thread;
- Le applicazioni sono **portabili**.

Thread operations latency (su un VAX, 1992)*

- **null fork**: time to *create*, *schedule*, *execute*, and *complete* a process/thread that invokes the "null" procedure;
- **signal-wait**: the time for a process/thread to *signal* a waiting process/thread and then *wait* on a condition;
- procedure call: 7 us; kernel trap: 17 us.

		(fast threads)	(Topaz threads)	(Ulrix processes)
	Operation	User-Level Threads	Kernel-Level Threads	Processes
creation cost →	Null Fork	34	948	11,300
synchronization cost →	Signal Wait	37	441	1,840

Table 4.1 Thread and Process Operation Latencies (μs) [ANDE92]

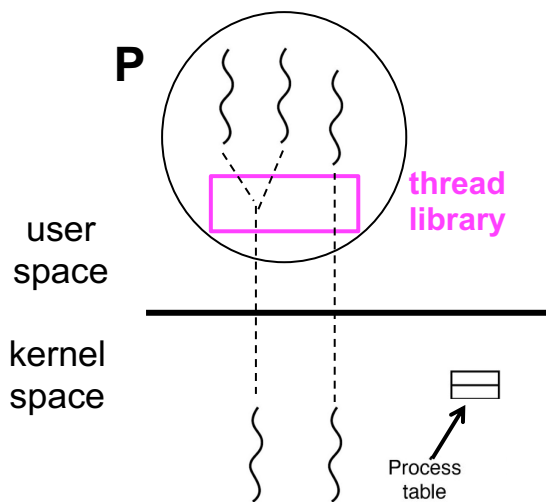
*Scheduler activations: *Effective Kernel Support for the User-level Management of Parallelism*, T. E. ANDERSON et al., **ACM Transactions on Computer Systems**, Vol. 10, No. 1, February 1992, Pages 53-79.

User-level thread (svantaggi)

- Un thread può *monopolizzare* la CPU:
 - a meno che non la rilasci **volontariamente**;
- In generale, nel caso **user-level**, c'è poco vantaggio nell'utilizzo del multithreading in sistemi multicore;
- Se un thread invoca una system call bloccante tutti i thread di quel processo si bloccano*. Opzioni:
 - rendere tutte le system call non bloccanti (impraticabile);
 - verificare se una system call genererà un blocco (richiede un **wrapper** "intorno" all'invocazione della system call).

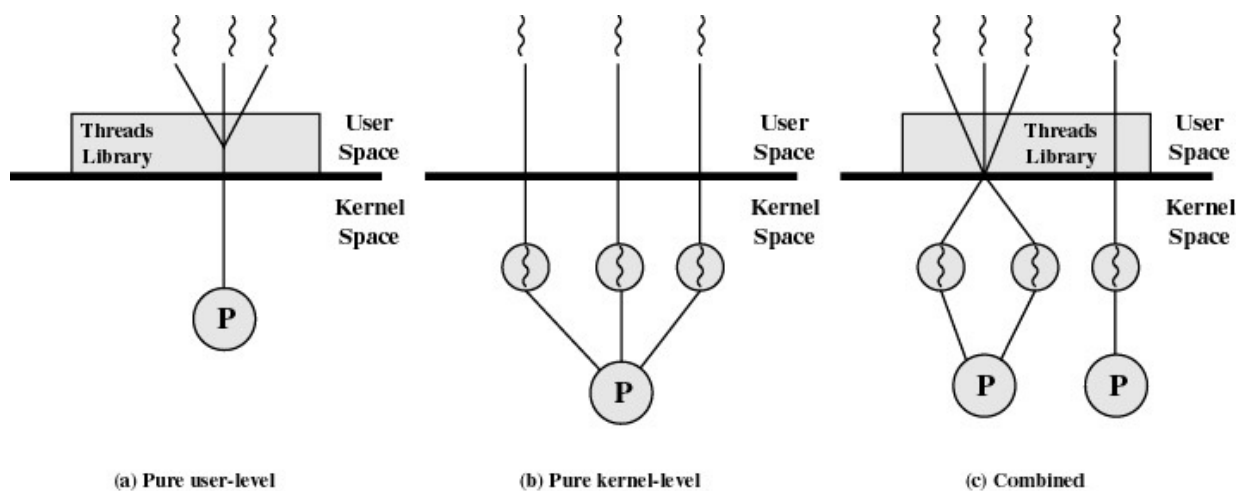
*tale problema non si pone per i kernel-level thread

Approccio ibrido (o combinato)



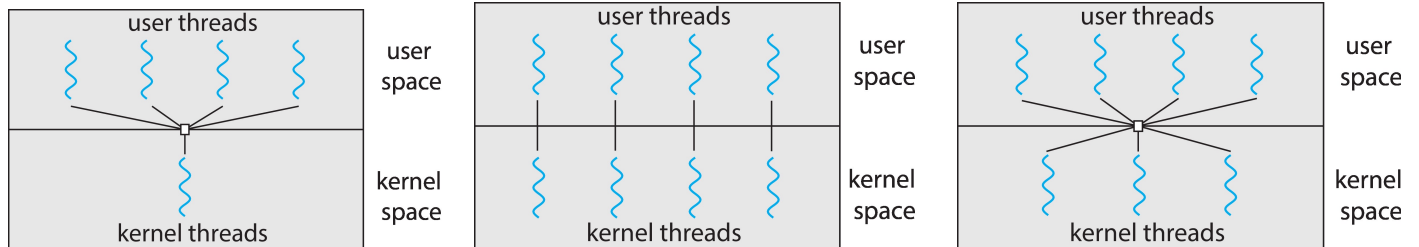
- Ai thread al livello d'utente corrispondono un numero di thread al livello del kernel *minore o uguale*:
- Il kernel è a conoscenza solo dei thread del kernel:
 - ogni thread del kernel ha un insieme di thread a livello utente che lo "usa", a turno.
- Il grosso dello **scheduling** è fatto nello spazio utente.

Rappresentazioni alternative

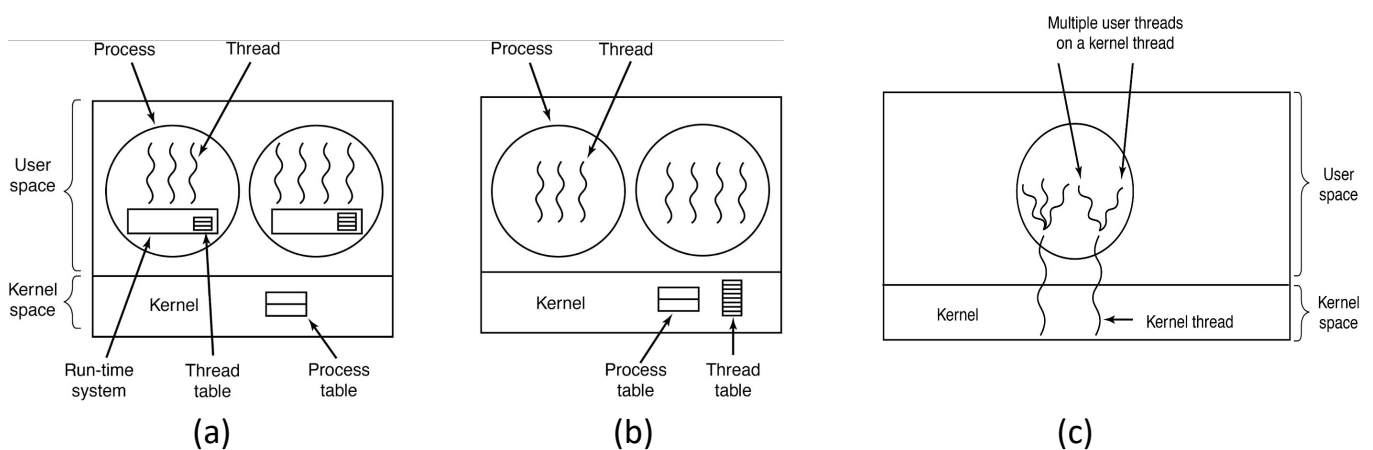


{ User-level thread
 (wavy circle) Kernel-level thread
 (P) Process

Rappresentazioni alternative



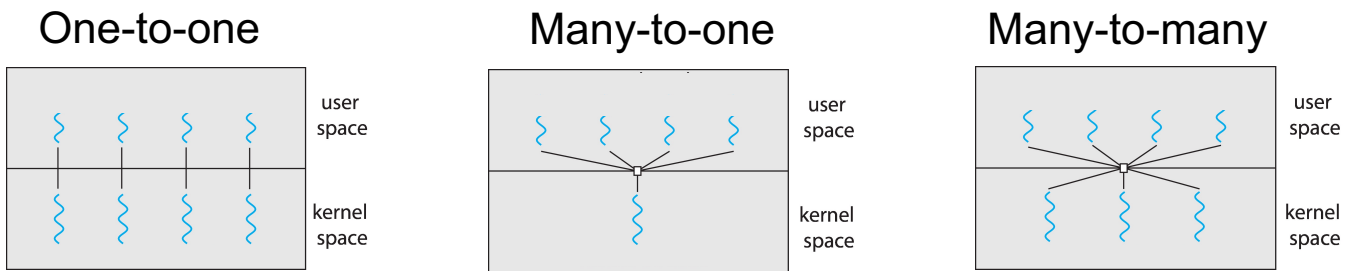
Rappresentazioni alternative*



(a) A user-level threads package. (b) A threads package managed by the kernel. (c) hybrid

*Adottata dal testo di riferimento.

Riepilogo dei modelli



- Ad oggi, i sistemi operativi general-purpose supportano i **thread kernel** (per es., Linux, Windows, Mac OS X);
- Il modello *one-to-one* è molto comune;
- Una **thread library** può essere implementata come libreria di thread interamente user space o con thread kernel supportati dal SO.

Un esempio: Pthreads

- **Standard** POSIX* per la creazione, sincronizzazione e gestione dei thread.
- E' una **specifica non una implementazione**:
 - implementazioni disponibili nei sistemi UNIX-like (per es. Linux);
 - forniti come user-level o kernel-level.

Thread call	Description
pthread_create	Create a new thread
pthread_exit	Terminate the calling thread
pthread_join	Wait for a specific thread to exit
pthread_yield	Release the CPU to let another thread run
pthread_attr_init	Create and initialize a thread's attribute structure
pthread_attr_destroy	Remove a thread's attribute structure

* Portable Operating System Interface for Unix (POSIX) – famiglia di standard IEEE relativi alla specifica dell'interfaccia del sistema operativo

Un esempio: Pthreads

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10
void * print_hello_world(void * tid)
{
    printf("Hello World. Greetings from thread %d\n", tid);
    pthread_exit(NULL);
}
int main(int argc, char * argv[])
{
    pthread_t threads[NUMBER_OF_THREADS];
    int status, i;
    for(i=0; i < NUMBER_OF_THREADS; i++) {
        status = pthread_create(&threads[i], NULL, print_hello_world, (void * )i);
        if (status != 0) {
            exit(-1);
        }
    }
    return 0;
}
```

